

Lab 6 - Connectionist Models

Part 1

```
import numpy as np
import matplotlib.pyplot as plt

class Perceptron:
    def __init__(self, num_inputs=2, learning_rate=0.2):
        self.learning_rate = learning_rate
        # Initialize weights randomly and bias to 0
        self.weights = np.random.rand(num_inputs) * 2 - 1
        self.bias = 0.0

    def _activation_function(self, x):
        # A simple step function: 1 if x >= 0, 0 otherwise
        return 1 if x >= 0 else 0

    def predict(self, inputs):
        # The core forward pass: sum of weighted inputs + bias
        total = np.dot(inputs, self.weights) + self.bias
        return self._activation_function(total)

# AND gate data
and_gate_data = [
    ([0, 0], 0), # input: [0, 0], target: 0
    ([0, 1], 0), # input: [0, 1], target: 0
    ([1, 0], 0), # input: [1, 0], target: 0
    ([1, 1], 1) # input: [1, 1], target: 1
]

# OR gate data
or_gate_data = [
    ([0, 0], 0), # input: [0, 0], target: 0
    ([0, 1], 1), # input: [0, 1], target: 1
    ([1, 0], 1), # input: [1, 0], target: 1
    ([1, 1], 1) # input: [1, 1], target: 1
]

def plot_points(data, ax):
    pts0 = np.array([x for x, t in data if t == 0])
    pts1 = np.array([x for x, t in data if t == 1])
    if len(pts0):
        ax.scatter(pts0[:, 0], pts0[:, 1], label="Class 0")
    if len(pts1):
        ax.scatter(pts1[:, 0], pts1[:, 1], label="Class 1")

def plot_decision_boundary(model, data, title):
    fig, ax = plt.subplots()
    plot_points(data, ax)

    # Draw decision boundary only if the second weight isn't (near) zero
    if abs(model.weights[1]) > 1e-12:
        xs = np.linspace(-0.2, 1.2, 100)
```

```

ys = -(model.weights[0] * xs) - model.bias) / model.weights[1]
ax.plot(xs, ys)

ax.set_xlim(-0.2, 1.2)
ax.set_ylim(-0.2, 1.2)
ax.set_xlabel("Input 1")
ax.set_ylabel("Input 2")
ax.legend()
ax.set_title(title)
plt.show()

# Main
p_and = Perceptron()
p_or = Perceptron()

# ----- Visualize AND -----
plot_decision_boundary(p_and, and_gate_data, "AND gate decision boundary")

# ----- Visualize OR -----
plot_decision_boundary(p_or, or_gate_data, "OR gate decision boundary")

```

Part 2

Part 2_1

Run the code. Does the perceptron successfully learn the AND gate logic? At what epoch does the error become zero?

Training for the AND gate

```

Epoch 1: Total error = 2
Epoch 2: Total error = 3
Epoch 3: Total error = 2
Epoch 4: Total error = 2
Epoch 5: Total error = 2
Epoch 6: Total error = 2
Epoch 7: Total error = 1
Epoch 8: Total error = 0

```

Training for the OR gate

```

Epoch 1: Total error = 2
Epoch 2: Total error = 2
Epoch 3: Total error = 2
Epoch 4: Total error = 2
Epoch 5: Total error = 2
Epoch 6: Total error = 1
Epoch 7: Total error = 0

```

Process finished with exit code 0

Yes, the perceptron learns the AND gate logic. In my run, the error reached zero after epoch 7. However the durations it took to reach zero were always different as the starting point (weights/bias in the beginning) was always different.

How do the weights and bias change during the training process? What do these changes represent?

The weights and bias change so that, with each epoch, the line more and more separates the blue dots from the orange dots. These changes represent the perceptron's effort to reduce its errors (wrong predictions) step by step.

Part 2_2

```
import numpy as np
import matplotlib.pyplot as plt
```

```
class Perceptron:
```

```
    def __init__(self, num_inputs=2, learning_rate=0.2):
        self.learning_rate = learning_rate
        # Initialize weights randomly and bias to 0
        self.weights = np.random.rand(num_inputs) * 2 - 1
        self.bias = 0.0
```

```
    def _activation_function(self, x):
        # A simple step function: 1 if x >= 0, 0 otherwise
        return 1 if x >= 0 else 0
```

```
    def predict(self, inputs):
        # The core forward pass: sum of weighted inputs + bias
        total = np.dot(inputs, self.weights) + self.bias
        return self._activation_function(total)
```

```
    def fit(self, data, epochs=20):
        for epoch in range(epochs):
            total_errors = 0
            for inputs, target in data:
                inputs = np.array(inputs)
                prediction = self.predict(inputs)
                error = target - prediction

                # Perceptron update rule
                self.weights += self.learning_rate * error * inputs
                self.bias += self.learning_rate * error

            total_errors += abs(error)

            print(f"Epoch {epoch + 1}: Total error = {total_errors}")
            if total_errors == 0:
                break
```

```
# AND gate data
and_gate_data = [
    ([0, 0], 0), # input: [0, 0], target: 0
    ([0, 1], 0), # input: [0, 1], target: 0
    ([1, 0], 0), # input: [1, 0], target: 0
```

```

    ([1, 1], 1) # input: [1, 1], target: 1
]

# OR gate data
or_gate_data = [
    ([0, 0], 0), # input: [0, 0], target: 0
    ([0, 1], 1), # input: [0, 1], target: 1
    ([1, 0], 1), # input: [1, 0], target: 1
    ([1, 1], 1) # input: [1, 1], target: 1
]

# XOR gate data
xor_gate_data = [
    ([0, 0], 0),
    ([0, 1], 1),
    ([1, 0], 1),
    ([1, 1], 0)
]

def plot_points(data, ax):
    pts0 = np.array([x for x, t in data if t == 0])
    pts1 = np.array([x for x, t in data if t == 1])
    if len(pts0):
        ax.scatter(pts0[:, 0], pts0[:, 1], label="Class 0")
    if len(pts1):
        ax.scatter(pts1[:, 0], pts1[:, 1], label="Class 1")

def plot_decision_boundary(model, data, title):
    fig, ax = plt.subplots()
    plot_points(data, ax)

    # Draw decision boundary only if the second weight isn't (near) zero
    if abs(model.weights[1]) > 1e-12:
        xs = np.linspace(-0.2, 1.2, 100)
        ys = -(model.weights[0] * xs) - model.bias / model.weights[1]
        ax.plot(xs, ys)

    ax.set_xlim(-0.2, 1.2)
    ax.set_ylim(-0.2, 1.2)
    ax.set_xlabel("Input 1")
    ax.set_ylabel("Input 2")
    ax.legend()
    ax.set_title(title)
    plt.show()

# Main
# print("Training for the AND gate\n")
# p_and = Perceptron()
# p_and.fit(and_gate_data)
# plot_decision_boundary(p_and, and_gate_data, "AND gate decision boundary")
#
# print("\nTraining for the OR gate\n")
# p_or = Perceptron()
# p_or.fit(or_gate_data)
# plot_decision_boundary(p_or, or_gate_data, "OR gate decision boundary")

# Create a new perceptron and train it on the XOR data
p_xor = Perceptron()

```

```
p_xor.fit(xor_gate_data)

print("\n--- Testing the XOR Perceptron ---")
for inputs, target in xor_gate_data:
    inputs_np = np.array(inputs)
    prediction = p_xor.predict(inputs_np)
    print(f"Input: {inputs}, Target: {target}, Prediction: {prediction}")
```

Why does the perceptron fail to learn the XOR gate? Connect this back to the lecture's concept of linear separability.

Because blue and orange cannot be separated by one straight line anymore.

Based on this limitation, what kind of model do you think would be necessary to solve the XOR problem? (Hint: Think about adding more layers).

A multi layered model could solve the problem, as it is able to form more complex shapes, like in our case using two linear lines to separate blue from orange.

Brainstorm some real-world problems that might be solved with a simple perceptron and some that would require a more complex network.

Real-world problems solved with a simple perceptron could be a temperature alert on a smartphone. When the temperature is higher than 30 °C it outputs “true” and gives an alert, otherwise it is “false.” Here, only one dimension for the decision is considered namely the value of the temperature.

More complex networks are already used in things like image classification. Here there are many dimensions (pixels, shapes or colors) to consider in order to make a decision, compared to having just a single value such as temperature.